

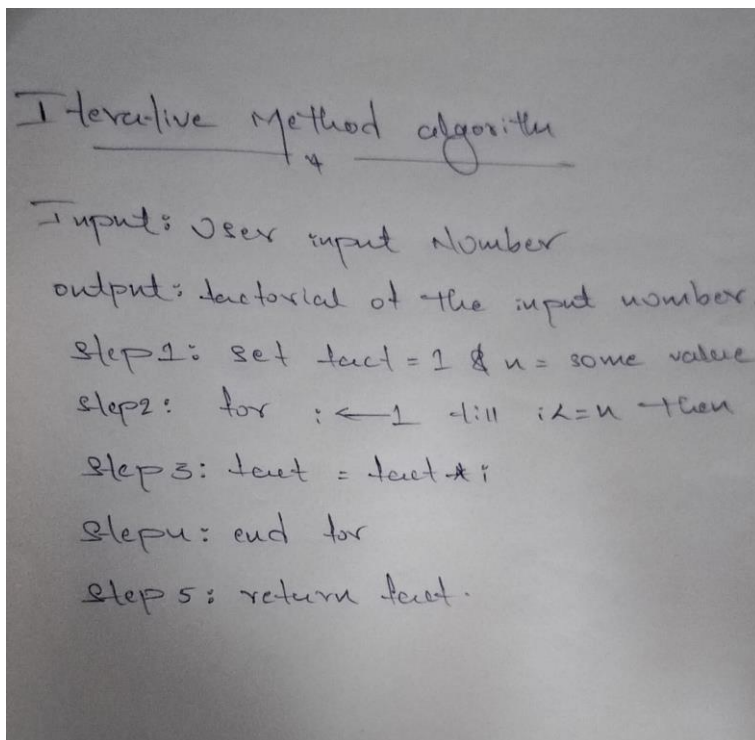
Practical: 4	To find the factorial of a given number using: 1. Iterative method 2. Recursive method
05/08/2026	

Aim:

To find the factorial of a given number using:

1. Iterative method
2. Recursive method

Algorithm:



Program:

RECURSIVE METHOD:

```
#include<iostream>
using namespace std;
int factorial(int n)
```

```
{  
  
if(n == 0 || n == 1)  
{  
    return 1;  
}  
  
return n * factorial(n - 1);  
}  
intmain()  
{  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
    cout << "Factorial = " << factorial(n);  
    return 0;  
}
```

Output

```
Enter a number: 10  
Factorial = 3628800  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:



Time complexity:

fact = 1; $\rightarrow 1$

if (num == 0) $\rightarrow 1$

return 1 $\rightarrow 1$

for (i = num; i > 1; i--) $\rightarrow 1 + 1 + 1 + \dots + 1$
 $2n + 1 = n$

fact = fact * i $\rightarrow n$

return fact $\rightarrow 1$

$= 2n + 1 + n$
 $= 3n + 1$
 $= n$

$T(n) = O(n)$

ITERATIVE METHOD:

ALGORITHM:

Recursive Method Algorithm:

Input: User input number
Output: Factorial number of input number

Step 1: start

Step 2: Read value of n

Step 3: factorial(n)

Step 4: check if n == 0 or n == 1

Return 1

Step 5: else
 $n * \text{factorial}(n-1)$

Step 6: print factorial

Step 7: stop.

PROGRAM:

```
#include<iostream>
using namespace std;
int factorial(int n)
{

if(n == 0 || n == 1)
{
return 1;
}
return n * factorial(n - 1);
}
intmain()
{
int n;
cout << "Enter a number: ";
cin >> n;
cout << "Factorial = " << factorial(n);
return 0;
}
```

OUTPUT:

Output

```
Enter a number: 10
Factorial = 3628800
```

```
=== Code Execution Successful ===
```

Time Complexity:



Time complexity:

```
int fact (int num) {  
    int fact = 1;   
    if (num == 0)   
        while (num >= 1)   
            num = --;   
            fact (num)   
    }  
    return fact  
}
```

$= 3n + 4$
 $= n$

Time complexity = $O(n)$

Best: $O(n)$
Avg: $O(n)$
Worst: $O(n)$

Conclusion:

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is $O(n)$, making them efficient for calculating factorials.